



Міністерство освіти і науки України
Національний технічний університет України «Київський політехнічний
інститут імені Ігоря Сікорського»
Факультет інформатики та обчислювальної техніки
Кафедра інформаційних систем та технологій

Лабораторна робота №7
З дисципліни «Технології розроблення програмного забезпечення»
Тема: **«ШАБЛОН «MEDIATOR», «FACADE», «BRIDGE», «TEMPLATE
METHOD»»**
Музичний програвач

Виконав:
Студент групи ІА-23
Волошин В. В.

Перевірив:
Мягкий М. Ю.

Київ-2024

Зміст

Тема:.....	3
Мета:.....	3
Хід роботи.....	3
1. Реалізувати не менше 3-х класів відповідно до обраної теми.....	3
2. Реалізувати один з розглянутих шаблонів за обраною темою.....	4
Висновки:.....	6
Код проекту:.....	6

Тема:

ШАБЛОН «MEDIATOR», «FACADE», «BRIDGE», «TEMPLATE METHOD»

Мета:

Ознайомитися з основними шаблонами проектування, такими як «Mediator», «Facade», «Bridge», «Template Method», дослідити їхні принципи роботи та навчитися використовувати для створення гнучкого та масштабованого програмного забезпечення, зокрема у розробці музичного програвача.

Хід роботи

1. Реалізувати не менше 3-х класів відповідно до обраної теми

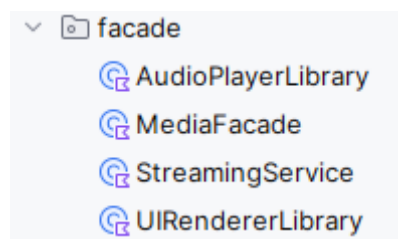


Рис. 1 — Структура проекту

У процесі виконання лабораторної роботи було реалізовано 4 класи, які забезпечують реалізацію функціоналу музичного програвача (рис. 1). Наведемо детальний опис кожного з цих класів:

1. **AudioPlayerLibrary**

Це низькорівневий клас, який відповідає за взаємодію з бібліотекою для відтворення аудіо. Він надає базові функціональні можливості для завантаження джерела аудіо (локальний файл або потік), запуску та зупинки відтворення музики. При цьому інкапсулює всі операції, пов'язані з аудіо, і забезпечує прямий доступ до функціоналу бібліотеки.

2. **UIRendererLibrary**

Клас для роботи з елементами інтерфейсу користувача. Він включає методи для оновлення інформації про поточний трек (назва, прогрес відтворення) та роботи з візуальними елементами програвача, наприклад, прогрес-баром. Забезпечує керування візуальним представленням музичного програвача і його інтерактивними компонентами.

3. **StreamingService**

Клас для завантаження та роботи з потоковою музикою. Він містить функції для підключення до віддаленого потоку через URL, буферизації

потоків даних, зміни джерела потоку для відтворення. При цьому інкапсулює всі процеси, пов'язані з потоковою передачею даних, забезпечуючи простий інтерфейс для роботи з потоковою музикою.

4. MediaFacade

Клас-фасад, який об'єднує роботу трьох попередніх класів (AudioPlayerLibrary, UIRendererLibrary, StreamingService) і надає простий у використанні API. Завдяки фасаді, клієнтський код може запускати відтворення локальних треків або потокової музики, зупиняти відтворення та оновлювати UI програвача (назва треку, прогрес відтворення). Таким чином абстрагує складність взаємодії з низькорівневими бібліотеками, спрощує клієнтський код і зменшує залежності, забезпечуючи єдиний інтерфейс для роботи з медіа.

2. Реалізувати один з розглянутих шаблонів за обраною темою



Рис. 2 — Діаграма класів

У проєкті додатку музичного програвача патерн Facade був реалізований для спрощення взаємодії з кількома спеціалізованими бібліотеками, які відповідають за аудіовідтворення, потокову передачу музики та відображення інтерфейсу. Для цього був створений клас MediaFacade, який об'єднує функціонал трьох низькорівневих компонентів: AudioPlayerLibrary, StreamingService та UIRendererLibrary.

Реалізація

- AudioPlayerLibrary надає функціонал для роботи з аудіо (завантаження файлів, запуск/зупинка відтворення).
- UIRendererLibrary відповідає за оновлення елементів інтерфейсу (назва треку, прогрес відтворення).
- StreamingService забезпечує завантаження і обробку поточкових джерел музики.

Клієнтський код не взаємодіє безпосередньо з цими класами, а використовує MediaFacade. Facade отримує виклики від клієнта, розподіляє їх між відповідними підсистемами та координує їхню роботу.

Проблеми, які вирішує патерн

- Складність інтеграції з низькорівневими бібліотеками: У проєкті використовуються різні бібліотеки для роботи з аудіо, потоками музики та візуальним представленням. Кожна з них має власний інтерфейс і складність у використанні. Завдяки Facade, клієнтський код працює лише з високорівневим інтерфейсом, не занурюючись у деталі реалізації.
- Зменшення залежностей між компонентами: Без фасаду клієнтський код мав би напряду працювати з декількома бібліотеками, що призвело б до високої зв'язаності системи. MediaFacade забезпечує централізовану точку доступу до функціоналу, ізолюючи клієнтський код від деталей роботи з бібліотеками.
- Зручність у використанні: Фасад спрощує виклик складних послідовностей дій (наприклад, запуск потокової музики з оновленням UI) до декількох простих методів. Це значно покращує зручність коду, робить його більш зрозумілим і легшим у підтримці.

Переваги використання

- Спрощення коду: Клієнтський код залишається мінімалістичним, оскільки всі складні операції інкапсульовані у фасаді.

- Легкість підтримки: Усі залежності від низькорівневих бібліотек зібрані в одному місці, що полегшує їх заміну чи модифікацію.
- Модульність: Взаємодія з окремими бібліотеками не впливає на загальний код додатку. Фасад забезпечує чітке розмежування відповідальностей.
- Зручність тестування: Фасад дозволяє легко писати модульні тести для клієнтського коду, замінюючи складні бібліотеки на прості mock-об'єкти.

Висновки:

У ході лабораторної роботи ми реалізували патерн Facade у контексті музичного програвача. Цей підхід дозволив спростити взаємодію з бібліотеками для аудіовідтворення, потокової передачі та оновлення інтерфейсу. Завдяки фасаду ми зменшили залежності клієнтського коду, підвищили його читабельність і забезпечили легкість розширення та підтримки системи.

Код проекту:

<https://github.com/vv0loshyn/TRPZ-labs>